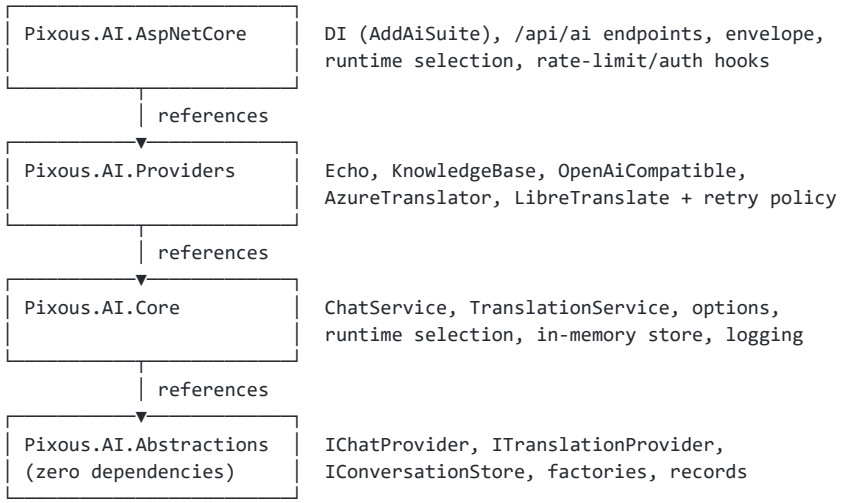
Architecture

Why this shape

Four .NET packages with strict one-way dependencies — consumers install only what they need, and no layer can reach back into its caller:



Angular mirrors it with one package (`@pixous/ai-suite`): `AiChatService`, `AiTranslationService`, `<pix-chat-widget>`, `<pix-language-switcher>`, auth interceptor — all talking to the backend BFF proxy only.

Key decisions

1. **Program against interfaces, swap via config.** `IChatProvider` / `ITranslationProvider` + factories give precedence per-request → runtime selection → configuration. Adding Anthropic/Google later = new adapter class + registration; core untouched (§7, providers.md).
2. **One OpenAI-compatible HTTP surface** instead of one SDK per vendor (OpenAI, Azure `/openai/v1`, Ollama, LM Studio, vLLM). No SDK version lock-in; zero third-party NuGet across all four packages.
3. **LLM stays monolingual; translation is a pipeline step** (BotBuilder multilingual-bot pattern). `ChatService` translates the reply when `responseLanguage` is requested.
4. **SSE streaming, no SignalR dependency.** `fetch` reader on the client, `text/event-stream` on the server — nothing extra to install either side.
5. **Stateless core, optional persistence.** `IConversationStore` seam with a bounded in-memory default; SQL/Mongo/Redis plug in without core changes.
6. **Secrets never cross a trust boundary.** Options bind non-secrets only; keys resolve from User Secrets / env / Key Vault at registration, are never logged, never returned (`/info` exposes booleans + config paths only).
7. **Standard envelope** `{ success, data, error{ code, message }, traceId }` on every JSON endpoint so all consumers handle errors one way.

Request flows

```
Angular widget → POST /api/ai/chat → ChatService → IChatProviderFactory
  → provider → (optional) Translate reply → envelope → widget
Angular switcher → TranslateService.use() (ngx-translate, instant) +
  POST /api/ai/translation/* for dynamic content
# panel → GET /api/ai/info|selection → POST /api/ai/selection (live switch)
```